WRITE-UP BABAK FINAL NCS 2026

Bina Darma Cyber Army

Universitas Bina Darma

Ahmad Ghumaydi

Agung syaputra

Bagaskara Amukti Palapa

29 September 2026

Write-Up Leaked Conversations Trail — Forensics

Kompetisi: Nusantara Cyber Security (NCS) 2026, Babak Final
Tanggal: 29 September 2026
Kategori: Forensics
Poin: 250
Berkas: `Leaked_Conversations_Trail.pcap`

Ringkasan

PCAP ini berisi banyak trafik pengalih perhatian dan percakapan HTTP. Petunjuk utama tersebar dalam tiga ruang obrolan awal: `liburan_squad`, `kelas_keamanan_siber`, dan `japri_G_A`. Empat komponen awal flag dikonfirmasi oleh lokasi dan dua paper yang dibagikan; komponen terakhir berasal dari pesan Base64. Percakapan lain memuat flag yang hampir benar, tetapi kata terakhirnya berbeda.

Tahapan analisis

1. `capinfos` menunjukkan 28.250 paket. `tshark -q -z io,phs` menemukan trafik HTTP dengan isi JSON. Saya memfilter HTTP `POST` dan mengambil `frame.number` serta `http.file_data`. Field kedua berupa heksadesimal; setelah diubah ke byte, isinya adalah objek JSON dengan kunci `room`, `sender`, dan `message`. Ada 345 pesan `POST`.

```
tshark -r Leaked_Conversations_Trail.pcap \
  -Y 'http.request.method == "POST"' \
  -T fields -e frame.number -e http.file_data
```

2. Saya mengurutkan pesan menurut nomor frame dan mengelompokkan berdasarkan `room`. Lima komponen berikut didukung oleh percakapan dan rujukan di dalamnya:

Komponen	Bukti dalam PCAP	Hasil
Negara	Frame 44–54 di `liburan_squad` menetapkan perjalanan di Indonesia.	`indonesia`
Tempat	Frame 84–184 membahas api biru, kawah, penambang belerang, dan koordinat `-8.058613,114.242233`. Koordinat tersebut cocok dengan Kawah Ijen.	`kawahijen`
Algoritma	Frame 214 menautkan paper pertama. Frame 254–274 menunjuk algoritma pelatihan MLP yang merambatkan galat ke belakang; paper menyebutnya *backpropagation*.	`backpropagation`
Tool	Frame 224 menautkan paper kedua tentang protokol Bitcoin. Frame 314–334 meminta nama tool penangkap paket dalam paper; tool tersebut adalah Wireshark.	`wireshark`
Kata terakhir	Frame 384 di `japri_G_A` berisi `YW1hbg==`. Decode Base64 menghasilkan `aman`, juga dikonfirmasi oleh balasan pada frame 404.	`aman`

3. Frame 3444 berisi `NCS{indonesia_kawahijen_backpropagation_wireshark_berhasil}`. Empat komponen awalnya cocok dengan bukti di atas dan memberi ejaan token `kawahijen`, tetapi kata `berhasil` bertentangan dengan pesan terkode pada `japri_G_A`. Saya mempertahankan empat komponen yang telah dikonfirmasi dan mengganti komponen terakhir menjadi `aman`.

Tools yang digunakan

Tool	Kegunaan
`capinfos`	Melihat ukuran dan jumlah paket.
`tshark`	Mengidentifikasi protokol dan mengekstrak isi HTTP `POST`.
Python 3	Membaca JSON, mendekode Base64, dan merakit flag.

Skrip reproduksi

Simpan kode ini sebagai `solve.py` di direktori yang sama dengan PCAP, lalu jalankan `python3 solve.py`. Skrip memerlukan `tshark` pada `PATH`. Interpretasi koordinat dan istilah paper dijelaskan pada tabel di atas; skrip memeriksa bukti dalam PCAP dan memakai ejaan empat komponen dari kandidat hampir benar.

```
#!/usr/bin/env python3
"""Extract the chat clues and assemble the flag from the PCAP."""

import base64
import json
import re
import subprocess
from pathlib import Path


pcap = Path(__file__).with_name("Leaked_Conversations_Trail.pcap")
capture = subprocess.run(
    [
        "tshark", "-r", str(pcap),
        "-Y", 'http.request.method == "POST"',
        "-T", "fields", "-e", "frame.number", "-e", "http.file_data",
    ],
    check=True,
    capture_output=True,
    text=True,
)

messages = {}
for line in capture.stdout.splitlines():
    number, payload_hex = line.split("\t", 1)
    messages[int(number)] = json.loads(bytes.fromhex(payload_hex))

assert len(messages) == 345
assert messages[44]["room"] == "liburan_squad"
assert "INDONESIA" in messages[54]["message"]
assert "-8.058613,114.242233" in messages[184]["message"]
assert "preprints.org" in messages[214]["message"]
assert "MLP" in messages[254]["message"]
assert "mecs-press.org" in messages[224]["message"]
assert "capture" in messages[324]["message"] or "tools" in messages[324]["message"]

# The papers and coordinate identify backpropagation, Wireshark, and Kawah Ijen.
# The near-match in frame 3444 confirms the four-token spelling and order.
decoy = re.search(r"NCS\{[^}]+\}", messages[3444]["message"]).group()
assert decoy == "NCS{indonesia_kawahijen_backpropagation_wireshark_berhasil}"

encoded = re.search(r"[A-Za-z0-9+/]+={0,2}", messages[384]["message"].split(": ", 1)
[1]).group()
last_word = base64.b64decode(encoded, validate=True).decode("ascii")
assert last_word == "aman" and messages[384]["room"] == "japri_G_A"

flag = decoy.rsplit("_", 1)[0] + "_" + last_word + "}"
print(flag)
```

Bukti hasil

```
$ python3 solve.py
```

```
NCS{indonesia_kawahijen_backpropagation_wireshark_aman}
```

Flag ini direkonstruksi dari bukti dalam PCAP dan rujukan paper. Tidak ada oracle penerimaan flag di dalam berkas challenge, sehingga pengujian ke platform CTF tidak termasuk dalam verifikasi lokal.

Flag


```
NCS{indonesia_kawahijen_backpropagation_wireshark_aman}
```

WRITE-UP CHALLENGE: REV - chall_3

Kompetisi: Nusantara Cyber Security (NCS) 2026 - Babak Final
Kategori: Reverse Engineering
Target File: `chall_3` (ELF 64-bit x86-64 Executable)
Tingkat Kesulitan: Intermediate

---

1. Ringkasan Eksekutif (Executive Summary)

Challenge `chall_3` menyajikan sebuah binary ELF 64-bit stripped yang mengimplementasikan proteksi anti-analisis dan validasi berbasis arsitektur multi-proses (Parent-Child) menggunakan teknik **Nanomites**.

Pada teknik ini, alur validasi input flag sengaja dipecah:

1. Proses **Child** bertindak sebagai antarmuka input dan memicu *software breakpoint* (`int3` / SIGTRAP) untuk setiap karakter input.
2. Proses **Parent** bertindak sebagai debugger (*tracer* via `ptrace`) yang menangani sinyal `SIGTRAP`, membaca isi register serta memori Child, melakukan operasi perbandingan XOR, dan memodifikasi register `RAX` Child untuk menentukan validitas karakter.

Melalui pembedahan statis disassembly instruksi x86-64 pada kedua fungsi tersebut, ditemukan dua blok array konstanta sepanjang 32 byte yang digunakan dalam operasi XOR. Dengan sifat involutori aljabar XOR ($A \oplus B = C \iff B = A \oplus C$), flag asli dapat diekstraksi secara deterministik tanpa perlu membypass ptrace secara dinamis.

Flag yang diperoleh adalah:

```
NCS{N4n0m1t3s_4r3_v3ry_4nn0y1ng}
```

---

2. Informasi File & Lingkungan (File & Environment Information)

A. Metadata Binary

Pemeriksaan awal menggunakan utilitas `file` dan `strings`:

```
$ file chall_3
chall_3: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=e536b27451b1b1ed80fa4bbde52378d196bf2ff9, for GNU/Linux 3.2.0, stripped
```

B. Karakteristik & Proteksi

- **Arsitektur:** x86-64 (64-bit Little Endian)
- **Format:** ELF PIE (Position Independent Executable)
- **Symbol Table:** Stripped (seluruh nama fungsi non-ekspor dan label dihapus)
- **Kompilasi:** GCC 13.3.0 (Ubuntu)

C. Tools yang Digunakan

Alat / Utilitas	Versi / Tipe	Fungsi Penggunaan
`file`	File Architecture Utility	Identifikasi tipe file, endianness, dan status stripped
`strings`	GNU Binutils	Identifikasi string UI, format string, dan import fungsi C
`objdump`	GNU Binutils (x86-64)	Disassembly seksi `.text` untuk merekonstruksi logika assembly
`python3`	Python 3.12 (struct, bytes)	Solver script untuk rekonstruksi data 64-bit dan kalkulasi XOR

Alat / Utilitas	Versi / Tipe	Fungsi Penggunaan
`bash`	Linux Shell	Eksekusi verifikasi flag langsung terhadap binary target

---

3. Tahapan Analisis (Step-by-Step Analysis)

Tahap 1: Inspeksi String & Import API

Melalui pemeriksaan string biner, diperoleh sejumlah informasi kritis:

```
$ strings chall_3
...
fgets
stdin
puts
perror
__stack_chk_fail
exit
fork
strlen
ptrace
strcspn
raise
printf
waitpid
========================================
      Nusantara Cyber Security (NCS)
Enter the flag:
Wrong length!
Correct! The flag is %s
Wrong!
...
```

Temuan Penting: Keberadaan fungsi `fork`, `ptrace`, `waitpid`, dan `raise` mengindikasikan adanya hubungan *tracer-tracee* (anti-debugging atau *process intercommunication*).

Tahap 2: Analisis Alur Eksekusi Utama (`main` - Offset `0x16b9`)

Disassembly fungsi `main` (`0x16b9 - 0x170b`):

```
16c5:  call   11b0 <fork@plt>
16ca:  mov    DWORD PTR [rbp-0x4], eax    ; simpan hasil fork
16cd:  cmp    DWORD PTR [rbp-0x4], 0x0
16d1:  jns    16e9                        ; jika >= 0 lanjut
...
16e9:  cmp    DWORD PTR [rbp-0x4], 0x0
16ed:  jne    16fb
16ef:  mov    eax, 0x0
16f4:  call   12a9                        ; PID == 0 -> Panggil Fungsi Child (0x12a9)
16f9:  jmp    1705
16fb:  mov    eax, DWORD PTR [rbp-0x4]
16fe:  mov    edi, eax                    ; Argumen edi = Child PID
1700:  call   1405                        ; PID > 0  -> Panggil Fungsi Parent (0x1405)
```

Alur program terbagi menjadi dua cabang independen:

- Child menjalankan subrutin pada offset `0x12a9`.
- Parent menjalankan subrutin pada offset `0x1405` dengan meneruskan PID Child sebagai argumen pertama (`edi`).

Tahap 3: Analisis Proses Child (Offset `0x12a9`)

1. **Inisialisasi Ptrace & Self-Stop:**

```
12dd:  call   ptrace@plt   ; ptrace(PTRACE_TRACEME, 0, 0, 0)
12e2:  mov    edi, 0x13    ; 0x13 = 19 = SIGSTOP
12e7:  call   raise@plt    ; raise(SIGSTOP) -> Menyerahkan kontrol awal ke Parent
```

2. **Input & Validasi Panjang:**

```
1340:  call   fgets@plt    ; Membaca input ke buffer [rbp-0x70] (maks 100 byte)
1365:  call   strcspn@plt  ; Menghapus newline '\n'
1376:  call   strlen@plt   ; Menghitung panjang input
137b:  cmp    rax, 0x20    ; Memeriksa apakah panjang == 32 (0x20)
137f:  je     139a
138b:  call   puts@plt     ; "Wrong length!"
1395:  call   exit@plt
```

Kondisi 1: Panjang flag harus tepat **32 karakter**.

3. **Loop Validasi Nanomites:**

```
139a:  mov    DWORD PTR [rbp-0x78], 1    ; is_valid = 1
13a1:  mov    DWORD PTR [rbp-0x74], 0    ; i = 0
...
; Loop Body:
13aa:  mov    edi, DWORD PTR [rbp-0x74]  ; edi = i
13ad:  lea    rsi, [rbp-0x70]           ; rsi = &input_buffer
13b1:  int3                             ; Trap ke Parent (SIGTRAP)
13b2:  test   eax, eax                  ; Memeriksa nilai kembalian RAX setelah resumed
13b4:  jne    13bf                      ; Jika eax != 0 -> Karakter benar, lanjut ke i
berikutnya
13b6:  mov    DWORD PTR [rbp-0x78], 0    ; Jika eax == 0 -> is_valid = 0
13bd:  jmp    13c9                      ; Break loop
13bf:  add    DWORD PTR [rbp-0x74], 1    ; i++
13c3:  cmp    DWORD PTR [rbp-0x74], 0x1f ; Loop sampai i = 31 (32 karakter)
13c7:  jle    13aa
```

Pada instruksi `int3` (`0x13b1`), eksekusi Child terhenti. Child mengandalkan Parent untuk memvalidasi `input_buffer[i]` dan memodifikasi nilai `RAX`.

Tahap 4: Analisis Proses Parent / Tracer (Offset `0x1405`)

1. **Penerimaan Sinyal Awal:**

```
1440:  call   waitpid@plt              ; Menangkap SIGSTOP dari Child
1457:  mov    edi, 7                   ; 7 = PTRACE_CONT
1461:  call   ptrace@plt               ; Melanjutkan Child hingga meminta input
```

2. **Inisialisasi Dua Buffer Kunci di Stack Parent:** Parent menyiapkan 2 array konstanta masing-masing berukuran 32 byte (`4 * 8-byte QWORD`):

Array 1 (`arr1`) di stack `[rbp-0x50]` hingga `[rbp-0x31]`:

- Offset `0x1466`: `0x7c6b5a4938271605` $\rightarrow$ `[rbp-0x50]`
- Offset `0x1470`: `0x04f3e2d1c0af9e8d` $\rightarrow$ `[rbp-0x48]`
- Offset `0x1482`: `0x8c7b6a5948372615` $\rightarrow$ `[rbp-0x40]`
- Offset `0x148c`: `0x1403f2e1d0bfae9d` $\rightarrow$ `[rbp-0x38]`

Array 2 (`arr2`) di stack `[rbp-0x30]` hingga `[rbp-0x11]`:

- Offset `0x149e`: `0x4c056e074374554b` $\rightarrow$ `[rbp-0x30]`
- Offset `0x14a8`: `0x76c7bda2f3dbafe0` $\rightarrow$ `[rbp-0x28]`
- Offset `0x14ba`: `0xb824132b7b417926` $\rightarrow$ `[rbp-0x20]`

- Offset `0x14c4`: `0x69649cd0a98fc0f3` $\rightarrow$ `[rbp-0x18]`

3. **Penanganan Trap `int3` dan Validasi Karakter:**

```
14ed:  call   waitpid@plt              ; Menangkap status Child
1531:  and    eax, 0xff00
1536:  cmp    eax, 0x500               ; Sinyal == SIGTRAP (5 << 8 = 0x500)
153b:  jne    1673

; Membaca register Child ke struct user_regs_struct di [rbp-0x130]
1558:  mov    edi, 0xc                 ; 0xc = 12 = PTRACE_GETREGS
1562:  call   ptrace@plt

; Ekstraksi RDI (offset +112 = [rbp-0xc0]) dan RSI (offset +104 = [rbp-0xc8])
1567:  mov    rax, QWORD PTR [rbp-0xc0] ; rax = regs.rdi (index i)
156e:  mov    DWORD PTR [rbp-0x144], eax
1574:  mov    rax, QWORD PTR [rbp-0xc8] ; rax = regs.rsi (alamat input_buffer)
157b:  mov    QWORD PTR [rbp-0x140], rax

; Membaca 1 byte karakter Child dari memori (input_buffer + i)
15af:  mov    eax, DWORD PTR [rbp-0x154]; child_pid
15bc:  mov    edi, 0x2                 ; 2 = PTRACE_PEEKDATA
15c6:  call   ptrace@plt
15d9:  mov    BYTE PTR [rbp-0x149], al  ; al = input_buffer[i]

; Logika Verifikasi XOR:
15e7:  movzx  eax, BYTE PTR [rbp+rax*1-0x50] ; eax = arr1[i]
15ec:  xor    al, BYTE PTR [rbp-0x149]       ; al = arr1[i] ^ input_buffer[i]
15f2:  mov    edx, eax
15fc:  movzx  eax, BYTE PTR [rbp+rax*1-0x30] ; eax = arr2[i]
1601:  cmp    dl, al                         ; Bandingkan: (arr1[i] ^ input_buffer[i]) ==
arr2[i]
1603:  jne    1612
1605:  movq   QWORD PTR [rbp-0xe0], 0x1      ; Jika Cocok: regs.rax = 1
1610:  jmp    162a
1612:  movq   QWORD PTR [rbp-0xe0], 0x0      ; Jika Beda : regs.rax = 0
1626:  ...

; Menyimpan kembali register RAX ke Child dan melanjutkan eksekusi
1641:  mov    edi, 0xd                 ; 0xd = 13 = PTRACE_SETREGS
164b:  call   ptrace@plt
1662:  mov    edi, 0x7                 ; 7 = PTRACE_CONT
166c:  call   ptrace@plt
```

---

4. Metode & Teknik Rekonstruksi (Methodology & Mathematical Proof)

A. Konsep Teknik Nanomites

Teknik **Nanomites** (atau *debug-driven code execution*) adalah teknik proteksi di mana instruksi percabangan atau komparasi kritis digantikan oleh exception/breakpoint (`int3`). Proses debugger induk mengontrol seluruh alur program dengan memeriksa state register serta memori tracee.

- Hal ini menggagalkan dekompilator standar (seperti Ghidra/IDA Pro) pada proses Child karena percabangan tampak mati di instruksi `int3`.
- Hal ini juga mencegah debugger eksternal (seperti GDB standar) melakukan *attach* karena Child sudah di-*trace* oleh Parent melalui `PTRACE_TRACEME`.

B. Rumusan Matematika Rekonstruksi

Dari disassembly baris `0x15e7 - 0x1601`, kondisi keberhasilan adalah: $$\text{arr1}[i] \oplus \text{flag}[i] = \text{arr2}[i]$$

Sesuai sifat aljabar Boolean operasi XOR ($\oplus$): $$\begin{aligned} (\text{arr1}[i] \oplus \text{flag}[i]) \oplus \text{arr1}[i] &= \text{arr2}[i] \oplus \text{arr1}[i] \ \text{flag}[i] \oplus (\text{arr1}[i] \oplus \text{arr1}[i]) &= \text{arr1}[i] \oplus \text{arr2}[i] \ \text{flag}[i] \oplus 0 &= \text{arr1}[i] \oplus \text{arr2}[i] \ \mathbf{flag}[i] &= \mathbf{arr1}[i] \oplus \mathbf{arr2}[i] \end{aligned}$$

Dengan demikian, karakter flag asli dapat direkonstruksi secara instan dengan melakukan XOR langsung antara byte `arr1` dan `arr2`.

---

5. Script Solver & Implementasi (Solver Script)

Berikut adalah script Python independen untuk merekonstruksi dan mendekode flag secara otomatis:

```
#!/usr/bin/env python3
"""
NCS 2026 Final - Reverse Engineering
Solver for chall_3 (Nanomites Reversal)
"""
import struct

def solve():
    # 4 buah QWORD (Little-Endian) untuk Array 1 (Stack [rbp-0x50])
    arr1_qwords = [
        0x7c6b5a4938271605,
        0x04f3e2d1c0af9e8d,
        0x8c7b6a5948372615,
        0x1403f2e1d0bfae9d
    ]

    # 4 buah QWORD (Little-Endian) untuk Array 2 (Stack [rbp-0x30])
    arr2_qwords = [
        0x4c056e074374554b,
        0x76c7bda2f3dbafe0,
        0xb824132b7b417926,
        0x69649cd0a98fc0f3
    ]

    # Konversi array 64-bit integer ke deretan 32 byte raw little-endian
    arr1_bytes = b"".join(struct.pack("<Q", val) for val in arr1_qwords)
    arr2_bytes = b"".join(struct.pack("<Q", val) for val in arr2_qwords)

    # Operasi XOR: flag[i] = arr1[i] ^ arr2[i]
    flag_bytes = bytes(b1 ^ b2 for b1, b2 in zip(arr1_bytes, arr2_bytes))
    flag = flag_bytes.decode("utf-8")

    print("[+] Flag Berhasil Direkonstruksi:")
    print(flag)
    return flag

if __name__ == "__main__":
    solve()
```

---

6. Hubungan Analisis dengan Flag yang Diperoleh (Analysis Correlation)

Tabel berikut membuktikan korelasi byte-by-byte hasil analisis statis dengan pembentukan string flag akhir:

Indeks i	Byte `arr1[i]` (Hex)	Byte `arr2[i]` (Hex)	Hasil XOR (`arr1 ^ arr2`)	Karakter ASCII
0	`0x05`	`0x4b`	`0x4e`	**N**
1	`0x16`	`0x55`	`0x43`	**C**
2	`0x27`	`0x74`	`0x53`	**S**
3	`0x38`	`0x43`	`0x7b`	**{**
4	`0x49`	`0x07`	`0x4e`	**N**
5	`0x5a`	`0x6e`	`0x34`	**4**
6	`0x6b`	`0x05`	`0x6e`	**n**
7	`0x7c`	`0x4c`	`0x30`	**0**

Indeks i	Byte `arr1[i]` (Hex)	Byte `arr2[i]` (Hex)	Hasil XOR (`arr1 ^ arr2`)	Karakter ASCII
8	`0x8d`	`0xe0`	`0x6d`	**m**
9	`0x9e`	`0xaf`	`0x31`	**1**
10	`0xaf`	`0xdb`	`0x74`	**t**
11	`0xc0`	`0xf3`	`0x33`	**3**
12	`0xd1`	`0xa2`	`0x73`	**s**
13	`0xe2`	`0xbd`	`0x5f`	**_**
14	`0xf3`	`0xc7`	`0x34`	**4**
15	`0x04`	`0x76`	`0x72`	**r**
16	`0x15`	`0x26`	`0x33`	**3**
17	`0x26`	`0x79`	`0x5f`	**_**
18	`0x37`	`0x41`	`0x76`	**v**
19	`0x48`	`0x7b`	`0x33`	**3**
20	`0x59`	`0x2b`	`0x72`	**r**
21	`0x6a`	`0x13`	`0x79`	**y**
22	`0x7b`	`0x24`	`0x5f`	**_**
23	`0x8c`	`0xb8`	`0x34`	**4**
24	`0x9d`	`0xf3`	`0x6e`	**n**
25	`0xae`	`0xc0`	`0x6e`	**n**
26	`0xbf`	`0x8f`	`0x30`	**0**
27	`0xd0`	`0xa9`	`0x79`	**y**
28	`0xe1`	`0xd0`	`0x31`	**1**
29	`0xf2`	`0x9c`	`0x6e`	**n**
30	`0x03`	`0x64`	`0x67`	**g**
31	`0x14`	`0x69`	`0x7d`	**}**

Seluruh 32 byte hasil kalkulasi membentuk kata yang koheren dalam Bahasa Inggris (*Leet speak*): `"N4n0m1t3s_4r3_v3ry_4nn0y1ng"` (Nanomites are very annoying), yang secara tematis menegaskan teknik proteksi yang digunakan dalam challenge ini.

---

7. Bukti Pendukung & Verifikasi (Supporting Evidence)

Pengujian langsung dilakukan pada lingkungan lokal terhadap file target `chall_3` dengan memberikan hasil flag yang telah direkonstruksi:

```
$ chmod +x chall_3
$ printf "NCS{N4n0m1t3s_4r3_v3ry_4nn0y1ng}\n" | ./chall_3
========================================
      Nusantara Cyber Security (NCS)
========================================
Enter the flag: Correct! The flag is NCS{N4n0m1t3s_4r3_v3ry_4nn0y1ng}
```

Hasil eksekusi binary mencetak string keberhasilan:

`"Correct! The flag is NCS{N4n0m1t3s_4r3_v3ry_4nn0y1ng}"`, membuktikan bahwa flag yang diekstraksi 100% valid dan lolos seluruh pemeriksaan pada proses Child dan Parent.

---

8. Flag Akhir

```
NCS{N4n0m1t3s_4r3_v3ry_4nn0y1ng}
```

WRITE-UP CHALLENGE: REV - chall_4

Kompetisi: Nusantara Cyber Security (NCS) 2026 - Babak Final
Kategori: Reverse Engineering
Target File: `chall_4` (ELF 64-bit x86-64 Executable)
Tingkat Kesulitan: Intermediate - Advanced

---

1. Ringkasan Eksekutif (Executive Summary)

Challenge `chall_4` menyajikan sebuah file biner ELF 64-bit x86-64 yang menerapkan teknik obfuscation berskala besar yang dikenal sebagai **Spaghetti Code** (atau *Straight-Line Unrolled Block Cipher*).

Karakteristik utama binary ini:

1. Ukuran seksi kode executable (`.text`) membengkak hingga $\approx 235\text{ KB}$ hanya untuk satu fungsi `main`.
2. Program menyematkan sebuah *decoy* / *fake flag* di dalam `.rodata` (`NCS{Th15_is_N0t_for_Y0U_Nu54nt4Ra}`) yang diperiksa menggunakan `strcmp`. Namun, validasi panjang string mewajibkan input tepat 32 karakter, sementara string decoy tersebut memiliki panjang 34 karakter sehingga cabang tersebut tidak akan pernah dapat tercapai secara valid.
3. Alur utama mengeksekusi tepat **15.000 blok transformasi sekuensial** tanpa percabangan (*straight-line code*) terhadap array state 32-byte.
4. Setiap operasi transformasi merupakan pemetaan bijektif (*one-to-one invertible mapping*) yang hanya terdiri dari 7 jenis primitif: `NOT`, `ADD`, `SUB`, dan `XOR` (baik terhadap konstanta maupun antar elemen state).
5. Pada akhir fungsi, seluruh 32 elemen state dibandingkan dengan nilai konstanta tertentu.

Dengan membedah struktur instruksi dan memanfaatkan sifat invertibilitas dari setiap operasi, nilai flag asli dapat dipulihkan secara instan melalui eksekusi mundur (*backward execution / inversion*) dari array target ke input awal.

Flag yang diperoleh adalah:

```
NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st
```

(Catatan: atau `NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st}` *dengan kurung penutup bila disesuaikan dengan konvensi platform).*

---

2. Informasi File & Lingkungan (File & Environment Information)

A. Metadata Binary

```
$ file chall_4
chall_4: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=293b5ccf073a3122a77c84ecee0e7578dbfd1302, for GNU/Linux 3.2.0, not stripped
```

B. Karakteristik Binary

- **Arsitektur:** x86-64 (64-bit Little Endian)
- **Format:** ELF PIE (Position Independent Executable)
- **Ukuran File:** 249.744 byte
- **Ukuran Seksi** `.text`**:** `0x39ebf` byte ($\approx 235\text{ KB}$)
- **Simbol:** *Not stripped* (simbol fungsi `main` tersedia)

C. Tools yang Digunakan

Alat / Utilitas	Fungsi Penggunaan
`file`	Identifikasi tipe format file dan arsitektur target
`strings`	Menemukan teks banner, pesan kesalahan, dan string decoy
`readelf`	Memeriksa tabel seksi dan ukuran seksi `.text` yang tidak wajar
`objdump`	Disassembly instruksi x86-64 fungsi `main` secara mendalam
`python3`	Parser instruksi assembly dan simulasi *backward inversion* untuk pemulihan flag
`bash`	Verifikasi dinamis hasil input flag langsung ke program

---

3. Tahapan Analisis Terstruktur (Step-by-Step Analysis)

Tahap 1: Analisis String & Identifikasi Decoy Flag

Pemeriksaan string pada file biner menghasilkan:

```
$ strings chall_4
...
================================================
        Nusantara Cyber Security (NCS)
Enter the flag:
Wrong length!
NCS{Th15_is_N0t_for_Y0U_Nu54nt4Ra}
Correct!
Wrong!
```

Ditemukan string yang menyerupai flag: `NCS{Th15_is_N0t_for_Y0U_Nu54nt4Ra}`. Namun, saat diuji langsung terhadap binary, program mengeluarkan respon `Wrong length!`.

Tahap 2: Analisis Logika Validasi Awal (`main` - Offset `0x1209`)

Disassembly awal fungsi `main` (`0x1209 - 0x1308`):

```
127b:  call   1100 <fgets@plt>       ; Membaca input ke buffer [rbp-0x70]
12a0:  call   10f0 <strcspn@plt>     ; Menghapus newline '\n'
12b1:  call   10c0 <strlen@plt>      ; Menghitung panjang string
12b6:  cmp    rax, 0x20              ; Cek apakah panjang == 32 (0x20)
12ba:  je     12d5
12bc:  lea    rax, [rip+0x39dc3]     ; "Wrong length!"
12c6:  call   10b0 <puts@plt>
12cb:  mov    eax, 1
12d0:  jmp    3afc9

; Jika panjang == 32:
12d5:  lea    rax, [rbp-0x70]
12d9:  lea    rdx, [rip+0x39db8]     ; String: "NCS{Th15_is_N0t_for_Y0U_Nu54nt4Ra}"
12e6:  call   1110 <strcmp@plt>      ; Komparasi langsung
12eb:  test   eax, eax
12ed:  jne    1308                   ; Jika tidak cocok, lompat ke logika transformasi
12f9:  call   10b0 <puts@plt>        ; "Correct!"
```

Analisis Decoy:

- Panjang string `NCS{Th15_is_N0t_for_Y0U_Nu54nt4Ra}` adalah **34 karakter**.
- Namun, instruksi pada `0x12b6` mengharuskan panjang input tepat **32 karakter**.
- Dengan demikian, kondisi `strcmp` pada `0x12e6` bersifat *dead code* / *honeypot* yang tidak akan pernah bernilai sama. Input yang sah akan selalu melompat ke offset `0x1308`.

Tahap 3: Analisis Seksi Transformasi (*Spaghetti Code*)

Pada offset `0x1308 - 0x1340`, program menyalin 32 byte input dari `[rbp-0x70]` ke dalam array kerja pada `[rbp-0x90]` s.d. `[rbp-0x71]`:

```
1314:  mov    eax, DWORD PTR [rbp-0x94]
131c:  movzx  eax, BYTE PTR [rbp+rax*1-0x70]
132b:  mov    BYTE PTR [rbp+rax*1-0x90], dl  ; Copy byte input ke state[i]
```

Mulai dari offset `0x1342` hingga `0x3ab71` (sepanjang $\approx 235\text{ KB}$ kode assembly), program mengeksekusi rantai operasi modifikasi in-place terhadap elemen-elemen array `[rbp-0x90]` s.d. `[rbp-0x71]`.

Pemeriksaan kontrol aliran (*control flow*) membuktikan:

- **Tidak ada instruksi percabangan (`jmp`, `jcc`, `call`)** sama sekali di antara `0x1342` dan `0x3ab71`.
- Total terdapat **54.092 baris instruksi assembly** yang terkelompok ke dalam tepat **15.000 blok operasi**.

Tahap 4: Klasifikasi Primitif Operasi

Melalui analisis struktural seluruh 15.000 blok, seluruh operasi dapat diklasifikasikan ke dalam 7 variasi pasti:

No	Pola Instruksi Assembly	Operasi Logika	Frekuensi Kemunculan

1	`movzx eax, [rbp-A]`
	`not eax`
	`mov [rbp-A], al`
		$\text{state}[A] = \sim \text{state}[A] \pmod{256}$	17

2	`movzx eax, [rbp-A]`
	`add eax, imm`
	`mov [rbp-A], al`
		$\text{state}[A] = (\text{state}[A] + \text{imm}) \pmod{256}$	1.480

3	`movzx eax, [rbp-A]`
	`sub eax, imm`
	`mov [rbp-A], al`
		$\text{state}[A] = (\text{state}[A] - \text{imm}) \pmod{256}$	1.418

4	`movzx eax, [rbp-A]`
	`xor eax, imm`
	`mov [rbp-A], al`
		$\text{state}[A] = \text{state}[A] \oplus \text{imm}$	2.993

5	`movzx edx, [rbp-A]`
	`movzx eax, [rbp-B]`
	`add eax, edx`
	`mov [rbp-A], al`
		$\text{state}[A] = (\text{state}[A] + \text{state}[B]) \pmod{256}$	3.028

6	`movzx eax, [rbp-A]`
	`movzx edx, [rbp-B]`
	`sub eax, edx`
	`mov [rbp-A], al`
		$\text{state}[A] = (\text{state}[A] - \text{state}[B]) \pmod{256}$	2.988

7	`movzx edx, [rbp-A]`
	`movzx eax, [rbp-B]`
	`xor eax, edx`
	`mov [rbp-A], al`
		$\text{state}[A] = \text{state}[A] \oplus \text{state}[B]$	3.076

Total			**15.000**

Karakteristik Matematis:

1. Variabel tujuan (`dest`) selalu merupakan variabel pertama yang dimuat, dan nilai variabel sumber (`src`) tidak berubah pada langkah tersebut.
2. Setiap operasi adalah **bijeksi pada ruang $\mathbb{Z}_{256}$**, sehingga tidak ada entropi atau informasi

yang hilang (*lossless transformation*).

Tahap 5: Ekstraksi Nilai Pembanding Akhir (`0x3ab71 - 0x3afb5`)

Setelah 15.000 operasi selesai, program membandingkan masing-masing 32 byte array dengan nilai konstanta:

```
3ab71:  movzx  eax, BYTE PTR [rbp-0x90] ; state[0]
3ab78:  cmp    al, 0x23
3ab7a:  je     3ab95
...
3af97:  movzx  eax, BYTE PTR [rbp-0x71] ; state[31]
3af9b:  cmp    al, 0xf5
3af9d:  je     3afb5
3afb5:  call   puts ("Correct!")
```

Nilai target yang diharapkan untuk $\text{state}[0 \dots 31]$ secara berurutan adalah:

```
target = [
    0x23, 0xdd, 0x71, 0xc3, 0xa2, 0xaf, 0x92, 0x59,
    0x5f, 0x59, 0xac, 0x82, 0x5f, 0x8c, 0x04, 0xa5,
    0x93, 0xac, 0xfe, 0x4b, 0x7c, 0x96, 0x2a, 0x54,
    0x8d, 0x7f, 0xc5, 0x0a, 0x51, 0xad, 0xbe, 0xf5
]
```

---

4. Metode & Teknik Rekonstruksi (Methodology)

A. Konsep Pembalikan Alur (Backward Inversion)

Karena transformasi berbentuk barisan fungsi bijektif: $$S_{15000} = (f_{14999} \circ f_{14998} \circ \dots \circ f_0)(S_0) = \text{target}$$

Maka nilai awal S_0 (flag) dapat dihitung dengan menerapkan fungsi invers dalam urutan terbalik: $$S_0 = (f_0^{-1} \circ f_1^{-1} \circ \dots \circ f_{14999}^{-1})(\text{target})$$

B. Definisi Operasi Invers:

1. $\text{Invers}(x = \sim x) \implies x = \sim x$
2. $\text{Invers}(x = x + k) \implies x = x - k \pmod{256}$
3. $\text{Invers}(x = x - k) \implies x = x + k \pmod{256}$
4. $\text{Invers}(x = x \oplus k) \implies x = x \oplus k$
5. $\text{Invers}(x = x + y) \implies x = x - y \pmod{256}$
6. $\text{Invers}(x = x - y) \implies x = x + y \pmod{256}$
7. $\text{Invers}(x = x \oplus y) \implies x = x \oplus y$

---

5. Script Solver Otomatis (`solve.py`)

Berikut implementasi lengkap script otomasi untuk mengekstrak dan membalikkan 15.000 instruksi:

```
#!/usr/bin/env python3
"""
NCS 2026 Final - Reverse Engineering
Solver for chall_4 (Spaghetti Code Reversal)
"""
import subprocess
import re
import os
```

```
def solve():
    binary_path = os.path.join(os.path.dirname(os.path.abspath(__file__)), "chall_4")

    # 1. Disassemble seksi transformasi (0x1342 - 0x3ab71)
    objdump_cmd = [
        "objdump", "-M", "intel", "-d",
        "--start-address=0x1342",
        "--stop-address=0x3ab71",
        binary_path
    ]
    raw_disasm = subprocess.check_output(objdump_cmd).decode("latin1")
    lines = raw_disasm.splitlines()

    insts = [l.split("\t")[2].strip() for l in lines if len(l.split("\t")) >= 3]

    # Kelompokkan instruksi per blok penulisan ke memori stack
    blocks = []
    current_block = []
    for inst in insts:
        current_block.append(inst)
        if re.search(r"mov\s+BYTE PTR \[rbp-0x[0-9a-f]+\],\s*al", inst):
            blocks.append(current_block)
            current_block = []

    print(f"[*] Parsed {len(blocks)} transformation blocks.")

    # 2. Parsing tipe operasi dan operan
    ops = []
    for b in blocks:
        dest_off = int(re.search(r"\[rbp-(0x[0-9a-f]+)\]", b[-1]).group(1), 16)
        dest_idx = 0x90 - dest_off
        if len(b) == 3:
            if "not" in b[1]:
                ops.append(("not", dest_idx, None))
            else:
                m_imm = re.search(r"(add|sub|xor)\s+eax,\s*(0x[0-9a-f]+)", b[1])
                op = m_imm.group(1)
                imm = int(m_imm.group(2), 16) & 0xff
                ops.append((op + "_imm", dest_idx, imm))
        elif len(b) == 4:
            src_off = int(re.search(r"\[rbp-(0x[0-9a-f]+)\]", b[1]).group(1), 16)
            src_idx = 0x90 - src_off
            op = b[2].split()[0]
            ops.append((op + "_reg", dest_idx, src_idx))

    # 3. Konstanta pembanding akhir
    target = [
        0x23, 0xdd, 0x71, 0xc3, 0xa2, 0xaf, 0x92, 0x59,
        0x5f, 0x59, 0xac, 0x82, 0x5f, 0x8c, 0x04, 0xa5,
        0x93, 0xac, 0xfe, 0x4b, 0x7c, 0x96, 0x2a, 0x54,
        0x8d, 0x7f, 0xc5, 0x0a, 0x51, 0xad, 0xbe, 0xf5
    ]

    # 4. Eksekusi mundur (Backward execution)
    state = list(target)
    for op_type, dest, arg in reversed(ops):
        if op_type == "not":
            state[dest] = (~state[dest]) & 0xff
        elif op_type == "add_imm":
            state[dest] = (state[dest] - arg) & 0xff
        elif op_type == "sub_imm":
            state[dest] = (state[dest] + arg) & 0xff
        elif op_type == "xor_imm":
            state[dest] = state[dest] ^ arg
        elif op_type == "add_reg":
            state[dest] = (state[dest] - state[arg]) & 0xff
        elif op_type == "sub_reg":
            state[dest] = (state[dest] + state[arg]) & 0xff
        elif op_type == "xor_reg":
            state[dest] = state[dest] ^ state[arg]

    flag = bytes(state).decode("utf-8")
    print(f"[+] Flag berhasil direkonstruksi: {flag}")
    return flag
```

```
if __name__ == "__main__":
    solve()
```

---

6. Hubungan Analisis dengan Flag yang Diperoleh

Hasil rekonstruksi menghasilkan string 32-byte:

```
NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st
```

Korelasi Konseptual:

1. **Tema Kalimat:** Kalimat leet-speak yang terbentuk adalah *"Spaghetti Code Kills AI Fast"*, yang secara spesifik merujuk pada taktik pembuat soal dalam membuat ribuan baris kode lurus (*spaghetti code*) guna menggagalkan atau memperlambat analisis otomatis oleh AI / symbolic execution engines.
2. **Kesesuaian Buffer:** Batasan `strlen` program pada baris `0x12b6` adalah tepat 32 (`0x20`). String `NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st` memiliki panjang tepat 32 karakter, sehingga pas dengan batas buffer yang dialokasikan oleh pembuat challenge.

---

7. Bukti Pendukung & Verifikasi (Supporting Evidence)

Hasil pengujian langsung terhadap biner target `chall_4`:

```
$ printf "NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st\n" | ./chall_4
==================================================
       Nusantara Cyber Security (NCS)
==================================================
Enter the flag: Correct!
```

Program mengembalikan pesan keberhasilan: `Correct!` dan keluar dengan status kode `0`, membuktikan bahwa flag yang diperoleh lolos seluruh 15.000 tahap transformasi dan sesuai dengan nilai target.

---

8. Flag Akhir

```
NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st
```

(Atau `NCS{Sp4gh3tt1_C0d3_K1lls_A1_F4st}`*)*

Write-Up My Chall 5 — Reverse Engineering

Kompetisi: Nusantara Cyber Security (NCS) 2026, Babak Final
Tanggal: 29 September 2026
Kategori: Reverse Engineering
Poin: 250
Deskripsi: “Mungkin menggunakan cara kuno...”
Berkas: `chall_5` (ELF 64-bit x86-64 PIE)

Ringkasan

Program menerima input sepanjang 33 byte. Sebelum validasi utama, program membandingkan input dengan banyak string berbentuk flag palsu. Validasi utama disamarkan sebagai *state machine* dengan 8.000 operasi pada byte dan 33 perbandingan akhir. Saya menelusuri urutan state, memodelkan perubahan setiap byte, lalu mencari nilai input yang menghasilkan konstanta pada setiap perbandingan.

Tahapan analisis dan penyelesaian

1. `file` menunjukkan bahwa `chall_5` adalah ELF 64-bit x86-64 PIE yang tidak *stripped*. `strings` menemukan 100 string berpola `NCS{Y0uR_fl4g_..._n0t_4_r34l_0n3}`. Setiap string tersebut panjangnya 35 byte, sedangkan instruksi `cmp rax, 0x21` pada `main` mensyaratkan tepat 33 byte. Karena pemeriksaan panjang dilakukan lebih dahulu, string itu tidak dapat lolos sebagai input.
2. `objdump` menunjukkan bahwa input disalin ke buffer lokal mulai `[rbp-0x70]`. Program mengisi variabel state pada `[rbp-0xe4]` dengan `0x3104a`, lalu mengarahkannya melalui dispatcher di `0x2842`. Dispatcher memakai perbandingan state dan tiga *jump table*. Penelusuran mulai dari state awal menghasilkan 8.000 blok transformasi, kemudian 33 blok `cmp al, ...`, dan akhirnya blok yang mencetak `Correct!`.
3. Setiap blok transformasi hanya mengubah satu byte menggunakan `add`, `sub`, `xor`, atau `not`; hasil byte dibatasi modulo 256. Pasangan instruksi `movzx`/`mov` yang mendahului operasi tersebut menulis kembali byte yang sama. Karena tidak ada ketergantungan antarbyte pada jalur eksekusi ini, untuk setiap posisi saya menghitung keluaran dari seluruh 256 kemungkinan nilai input. Nilai yang menghasilkan konstanta pada blok `cmp` adalah byte flag di posisi tersebut. Setiap posisi menghasilkan tepat satu kandidat.

Tools yang digunakan

Tool	Kegunaan
`file` dan `strings`	Mengidentifikasi format program serta string pengecoh.
`objdump`	Memeriksa pemeriksaan panjang, dispatcher, blok transformasi, dan perbandingan akhir.
Python 3 dan Capstone 5.0.7	Menelusuri instruksi, memodelkan transformasi byte, dan merekonstruksi input.

Skrip reproduksi

Simpan kode berikut sebagai `solve.py` di direktori yang sama dengan `chall_5`. Jalankan `python3 solve.py`; dependensinya adalah `capstone` (`pip install capstone`). Skrip membaca binary asli, merekonstruksi flag, lalu menjalankan binary untuk memastikan hasilnya diterima.

```
#!/usr/bin/env python3
"""Recover the flag from chall_5's flattened byte-wise checker."""
from pathlib import Path
import re
import struct
```

```
import subprocess

from capstone import Cs, CS_ARCH_X86, CS_MODE_64


ROOT = Path(__file__).resolve().parent
BINARY = ROOT / "chall_5"
blob = BINARY.read_bytes()
md = Cs(CS_ARCH_X86, CS_MODE_64)
insns = list(md.disasm(blob[0x1120:0x801c0], 0x1120))
by_addr = {ins.address: i for i, ins in enumerate(insns)}

# The dispatcher is a search tree over the state in [rbp-0xe4].
states = {}
tables = {0x412a7: (0xcd024, 0x82044, 0x1e),
          0x412de: (0x89b2e, 0x820c0, 0x22),
          0x41315: (0x2c4b3, 0x8214c, 0x33)}
for i, ins in enumerate(insns[:-1]):
    match = re.fullmatch(r"dword ptr \[rbp - 0xe4\], (0x[0-9a-f]+|[0-9]+)", ins.op_str)
    if ins.mnemonic == "cmp" and match and insns[i + 1].mnemonic == "je":
        states[int(match.group(1), 0)] = int(insns[i + 1].op_str, 16)


def dispatch(state):
    if state in states:
        return states[state]
    pos = by_addr[0x2842]
    prev = 0
    for _ in range(100000):
        ins = insns[pos]
        op = ins.op_str
        if ins.address in tables:
            start, table, max_index = tables[ins.address]
            index = state - start
            if not 0 <= index <= max_index:
                raise RuntimeError(f"Bad table state: {state:x}")
            target = table + struct.unpack_from("<i", blob, table + 4 * index)[0]
            states[state] = target
            return target
        if ins.mnemonic == "cmp" and op.startswith("dword ptr [rbp - 0xe4], "):
            prev = state - int(op.split(", ")[1], 0)
        elif ins.mnemonic in {"je", "jne", "jg", "jge", "jl", "jle", "ja", "jae", "jb",
"jbe"}:
            taken = {"je": prev == 0, "jne": prev != 0, "jg": prev > 0,
                     "jge": prev >= 0, "jl": prev < 0, "jle": prev <= 0,
                     "ja": prev > 0, "jae": prev >= 0, "jb": prev < 0,
                     "jbe": prev <= 0}[ins.mnemonic]
            if taken:
                pos = by_addr[int(op, 16)]
                continue
        elif ins.mnemonic == "jmp":
            pos = by_addr[int(op, 16)]
            continue
        else:
            states[state] = ins.address
            return ins.address
        pos += 1
    raise RuntimeError(f"Dispatch loop for {state:x}")


def block_for(state):
    pos = by_addr[dispatch(state)]
    block = []
    while True:
        ins = insns[pos]
        if ins.mnemonic == "mov" and ins.op_str.startswith("dword ptr [rbp - 0xe4], "):
            nxt = int(ins.op_str.split(", ")[1], 0)
            return block, nxt
        block.append(ins)
        pos += 1
        if pos >= len(insns) or len(block) > 100:
            raise RuntimeError(f"Unrecognized block at state {state:x}")
```

```
def slot(op):
    m = re.fullmatch(r"byte ptr \[rbp - (0x[0-9a-f]+)\]", op)
    if m:
        off = int(m.group(1), 16)
        if 0x50 <= off <= 0x70:
            return 0x70 - off
    return None


trace = []
state = 0x3104a
seen = set()
while state and state not in seen:
    seen.add(state)
    block, nxt = block_for(state)
    trace.append((state, block, nxt))
    state = nxt

assert len(trace) == 8034 and state == 0

# Track the result of every possible input byte independently. The first
# movzx/mov pair in each block is an identity operation; the second transforms
# one byte with add, sub, xor or not.
maps = [list(range(256)) for _ in range(33)]
for _, block, _ in trace[:8000]:
    assert len(block) == 5
    target = slot(block[2].op_str.split(", ")[1])
    assert target is not None and target == slot(block[4].op_str.split(", ")[0])
    op = block[3]
    if op.mnemonic == "not":
        maps[target] = [(~v) & 255 for v in maps[target]]
    else:
        arg = int(op.op_str.split(", ")[1], 0)
        if op.mnemonic == "add":
            maps[target] = [(v + arg) & 255 for v in maps[target]]
        elif op.mnemonic == "sub":
            maps[target] = [(v - arg) & 255 for v in maps[target]]
        elif op.mnemonic == "xor":
            maps[target] = [(v ^ arg) & 255 for v in maps[target]]
        else:
            raise RuntimeError(f"Unexpected transform: {op}")

answer = [None] * 33
for _, block, _ in trace[8000:-1]:
    index = slot(block[0].op_str.split(", ")[1])
    cmp_ins = block[1]
    assert cmp_ins.mnemonic == "cmp" and cmp_ins.op_str.startswith("al, ")
    expected = int(cmp_ins.op_str.split(", ")[1], 0)
    candidates = [v for v, result in enumerate(maps[index]) if result == expected]
    assert len(candidates) == 1
    answer[index] = candidates[0]

flag = bytes(answer)
result = subprocess.run(["/lib64/ld-linux-x86-64.so.2", str(BINARY)],
                        input=flag + b"\n", capture_output=True, check=False)
assert result.returncode == 0 and b"Correct!" in result.stdout
print(flag.decode())
```

Bukti hasil

Keluaran `python3 solve.py`:

```
NCS{Th1s_1S_wh4T_H3ll_L00k5_L1k3}
```

Skrip juga mengirim flag tersebut ke `chall_5`. Proses berakhir dengan kode `0` dan mencetak `Correct!`. Kedua pemeriksaan itu dilakukan oleh `assert` sebelum flag ditampilkan. Dengan demikian, flag di bawah berasal dari hasil pembalikan transformasi dan telah diterima oleh binary challenge.

Flag

```
NCS{Th1s_1S_wh4T_H3ll_L00k5_L1k3}
```

My Chall 6 — Reverse Engineering

Kompetisi: NCS 2026, Babak Final (29 September 2026)
Kategori: REV
Deskripsi: “Jangan percaya dengan asisten Mu...”
Berkas challenge: `chall_6`
Flag: `NCS{Pr0mpt_1nj3ct10n_4nd_4dv3rs4r14l_C0d3_D3str0y5_LLMs}`

Ringkasan

Binary ELF menerima input sepanjang 56 byte dan memeriksanya setelah 25.000 operasi aritmetika byte. Sebanyak 6.351 operasi hanya mengubah tabel kerja 500 byte yang nilai awalnya sudah diketahui; 18.649 operasi lainnya mengubah input memakai nilai dari tabel itu. Karena tabel kerja tidak bergantung pada input, nilainya dapat disimulasikan maju, lalu perubahan pada input dibalik dari nilai pembanding akhir.

Tahapan analisis dan penyelesaian

1. `file` mengenali `chall_6` sebagai ELF x86-64 PIE. Dengan `objdump`, terlihat `main` memanggil `fgets`, menghapus newline, lalu membandingkan `strlen(input)` dengan `0x38` (56). Input disalin ke array lokal 56 byte pada `[rbp-0x270]` sampai `[rbp-0x239]`. Sebuah tabel kerja pada `[rbp-0x200]` sampai `[rbp-0x0d]` diisi dengan nilai `i & 0xff` untuk `i = 0..499`.
2. Disassembly dari offset `0x1334` sampai `0x78c41` berisi 25.000 perubahan byte menggunakan `add`, `sub`, `xor`, dan `not`. Parsing operand dengan Capstone menunjukkan bahwa semua operasi yang menulis tabel kerja hanya membaca tabel kerja. Operasi yang menulis input hanya membaca byte input tujuan dan nilai tabel kerja atau konstanta. Karena tidak ada perubahan tabel kerja yang dipengaruhi input, nilai tabel saat setiap operasi input dapat dihitung langsung.
3. Mulai offset `0x78c41`, program membandingkan 56 byte input hasil transformasi dengan konstanta. Skrip membaca konstanta tersebut, menghitung tabel kerja secara maju, dan mencatat operasi input beserta nilai operandnya. Kemudian operasi input dibalik dalam urutan terbalik dengan aritmetika modulo 256. Satu pembandingan memakai `test al, al`, sehingga nilai target byte itu adalah `0`. Hasil pembalikan adalah flag di atas.

Skrip reproduksi

Jalankan dari folder challenge dengan `python3 solve.py`. Dependensi: `capstone` (`pip install capstone`). Binary tidak perlu diberi izin eksekusi karena skrip memanggilnya melalui loader Linux untuk verifikasi.

```
from pathlib import Path
import subprocess

from capstone import CS_ARCH_X86, CS_MODE_64, Cs
from capstone.x86 import X86_OP_IMM, X86_OP_MEM, X86_OP_REG


binary = Path(__file__).with_name("chall_6").read_bytes()
decoder = Cs(CS_ARCH_X86, CS_MODE_64)
decoder.detail = True


def location(operand):
    assert operand.type == X86_OP_MEM
    offset = -operand.mem.disp
    if 0x239 <= offset <= 0x270:
        return "input", 0x270 - offset
    if 0xD <= offset <= 0x200:
        return "scratch", 0x200 - offset
    raise ValueError(hex(offset))
```

```
scratch = [i & 255 for i in range(500)]
input_operations = []
registers = {}
pending = None

for ins in decoder.disasm(binary[0x1334:0x78C41], 0x1334):
    operands = ins.operands
    if ins.mnemonic == "movzx":
        registers[ins.reg_name(operands[0].reg)] = location(operands[1])
    elif ins.mnemonic in ("add", "sub", "xor", "not"):
        assert pending is None
        left = registers["eax"]
        if ins.mnemonic == "not":
            right = None
        elif operands[1].type == X86_OP_IMM:
            right = ("constant", operands[1].imm & 255)
        else:
            assert operands[1].type == X86_OP_REG
            right = registers[ins.reg_name(operands[1].reg)]
        pending = ins.mnemonic, left, right
    elif ins.mnemonic == "mov":
        assert pending is not None
        target = location(operands[0])
        kind, left, right = pending

        def value(source):
            return source[1] if source[0] == "constant" else scratch[source[1]]

        if target[0] == "scratch":
            assert left[0] == "scratch"
            assert right is None or right[0] in ("scratch", "constant")
            result = scratch[left[1]]
            other = None if right is None else value(right)
            if kind == "add":
                result += other
            elif kind == "sub":
                result -= other
            elif kind == "xor":
                result ^= other
            else:
                result = ~result
            scratch[target[1]] = result & 255
        else:
            assert (left[0] == "input") != (right is not None and right[0] == "input")
            other = None if right is None else value(right) if right[0] != "input" else
value(left)
            input_operations.append((target[1], kind, left == target, other))

        registers = {}
        pending = None
    else:
        raise ValueError((hex(ins.address), ins.mnemonic, ins.op_str))

assert pending is None and len(input_operations) == 18649

expected = [None] * 56
current = None
for ins in decoder.disasm(binary[0x78C41:0x790C7], 0x78C41):
    if ins.mnemonic == "movzx" and ins.operands[1].type == X86_OP_MEM:
        current = location(ins.operands[1])
    elif ins.mnemonic == "cmp" and ins.operands[1].type == X86_OP_IMM:
        assert current[0] == "input"
        expected[current[1]] = ins.operands[1].imm & 255
    elif ins.mnemonic == "test" and ins.op_str == "al, al":
        assert current[0] == "input"
        expected[current[1]] = 0

assert all(x is not None for x in expected)
state = expected[:]
for target, kind, input_on_left, other in reversed(input_operations):
    result = state[target]
    if kind == "not":
        state[target] = ~result & 255
    elif kind == "xor":
```

```
        state[target] = result ^ other
    elif kind == "add":
        state[target] = (result - other) & 255
    elif input_on_left:
        state[target] = (result + other) & 255
    else:
        state[target] = (other - result) & 255

answer = bytes(state)
print(answer.decode())
process = subprocess.run(
    ["/lib64/ld-linux-x86-64.so.2", str(Path(__file__).with_name("chall_6"))],
    input=answer + b"\n", capture_output=True,
)
print(process.stdout.decode())
assert process.returncode == 0 and b"Correct!" in process.stdout
```

Bukti verifikasi

Skrip menghasilkan flag dan output binary berikut:

```
NCS{Pr0mpt_1nj3ct10n_4nd_4dv3rs4r14l_C0d3_D3str0y5_LLMs}
==================================================
        Nusantara Cyber Security (NCS)
==================================================
Enter the flag: Correct!
```

Flag

```
NCS{Pr0mpt_1nj3ct10n_4nd_4dv3rs4r14l_C0d3_D3str0y5_LLMs}
```

My Chall 1 — Crypto/Steganografi

Kompetisi: NCS 2026, Babak Final (29 September 2026)
Berkas challenge: `chall_1.png`
Flag: `NCS{mult1_l4y3r_st3g0_1s_fun_r1ght}`

Ringkasan

Gambar PNG tampak polos, tetapi menyimpan petunjuk dalam metadata dan arsip ZIP yang disisipkan setelah chunk `IEND`. Arsip berisi puisi dengan pesan pada spasi/tab di akhir baris dan sebuah berkas WAV. Pesan dari puisi menjadi kunci XOR untuk bit tersembunyi pada sampel audio berindeks prima.

Tahapan analisis dan penyelesaian

1. Pemeriksaan `chall_1.png` dengan `file`, `exiftool`, dan `xxd` menunjukkan PNG RGB berukuran 200 × 200 piksel. Pemeriksaan piksel dengan Python/Pillow menunjukkan seluruh piksel bernilai `(20, 20, 20)`, sehingga isi visualnya tidak memuat informasi. `exiftool` menampilkan metadata `Hint` dan peringatan adanya data setelah chunk `IEND`. Nilai `Hint` dibalik lalu didekode dari Base64, menghasilkan `XOR with 0x7A to reveal the archive`. Data setelah `IEND` di-XOR dengan `0x7A`; awal hasilnya adalah `PK\x03\x04`, yaitu header ZIP.
2. ZIP berisi `poem.txt` dan `noise.wav`. Pada setiap baris puisi, spasi dan tab setelah teks biasa dipetakan menjadi bit `0` dan `1`. Seluruh bit digabungkan per delapan bit, menghasilkan `Pr1m3_N01s3`. Isi puisi juga menyebut *silent sound* dan *prime directives*, yang mengarah ke WAV dan indeks prima.
3. `noise.wav` berisi 1.911 sampel mono 8-bit. Dengan indeks sampel dimulai dari nol, ambil bit paling rendah dari sampel pada indeks prima (`2, 3, 5, 7, ...`). Gabungkan bit menjadi byte, lalu XOR dengan kunci `Pr1m3_N01s3` secara berulang. Hasilnya diawali `NCS{mult1_l4y3r_st3g0_1s_fun_r1ght}`. Masih ada data setelah kurung kurawal penutup, sehingga flag diambil sampai `}`.

Skrip reproduksi

Jalankan dengan `python3` dari direktori yang berisi `chall_1.png`. Skrip ini hanya menggunakan pustaka standar Python dan langsung membaca berkas asli.

```
import base64
import io
import re
import struct
import wave
import zipfile
from pathlib import Path


png = Path("chall_1.png").read_bytes()
assert png.startswith(b"\x89PNG\r\n\x1a\n")

# Baca chunk PNG untuk menemukan metadata dan posisi akhir IEND.
offset = 8
hint = None
trailer_start = None
while offset < len(png):
    length = struct.unpack_from(">I", png, offset)[0]
    kind = png[offset + 4 : offset + 8]
    data = png[offset + 8 : offset + 8 + length]
    offset += 12 + length
    if kind == b"tEXt" and data.startswith(b"Hint\x00"):
        hint = data.split(b"\x00", 1)[1]
    if kind == b"IEND":
        trailer_start = offset
        break
```

```
assert hint is not None and trailer_start is not None
decoded_hint = base64.b64decode(hint[::-1]).decode()
print("Hint:", decoded_hint)
assert decoded_hint == "XOR with 0x7A to reveal the archive"

archive = bytes(byte ^ 0x7A for byte in png[trailer_start:])
assert archive.startswith(b"PK\x03\x04")
with zipfile.ZipFile(io.BytesIO(archive)) as zf:
    poem = zf.read("poem.txt")
    audio = zf.read("noise.wav")

# Spasi = 0, tab = 1; gabungkan whitespace di akhir setiap baris.
trailing = b"".join(re.search(rb"[ \t]+$", line).group()
                    for line in poem.splitlines())
key_bits = "".join("1" if char == 9 else "0" for char in trailing)
assert len(key_bits) % 8 == 0
key = bytes(int(key_bits[i : i + 8], 2)
            for i in range(0, len(key_bits), 8))
print("Kunci:", key.decode())

with wave.open(io.BytesIO(audio), "rb") as wav:
    assert wav.getnchannels() == 1 and wav.getsampwidth() == 1
    samples = wav.readframes(wav.getnframes())

# Saringan Eratosthenes untuk indeks prima berbasis nol.
prime = [True] * len(samples)
prime[0:2] = [False, False]
for n in range(2, int(len(samples) ** 0.5) + 1):
    if prime[n]:
        for multiple in range(n * n, len(samples), n):
            prime[multiple] = False

bits = "".join(str(samples[i] & 1)
               for i in range(len(samples)) if prime[i])
ciphertext = bytes(int(bits[i : i + 8], 2)
                   for i in range(0, len(bits) - 7, 8))
plaintext = bytes(byte ^ key[i % len(key)]
                  for i, byte in enumerate(ciphertext))
flag = re.search(rb"NCS\{[^}]+\}", plaintext)
assert flag is not None
print("Flag:", flag.group().decode())
```

Keluaran yang memverifikasi rantai penyelesaian:

```
Hint: XOR with 0x7A to reveal the archive
Kunci: Pr1m3_N01s3
Flag: NCS{mult1_l4y3r_st3g0_1s_fun_r1ght}
```

Flag

```
NCS{mult1_l4y3r_st3g0_1s_fun_r1ght}
```

My Chall 2 — Crypto/Steganografi

Kompetisi: NCS 2026, Babak Final (29 September 2026)
Berkas challenge: `chall_2.png`
Flag: `NCS{t0ur1ng_th3_gr4ff1t1_m4z3_w1th_m4th}`

Ringkasan

PNG menyimpan ZIP berpassword setelah chunk `IEND`. String berbentuk flag pada metadata PNG merupakan password ZIP. Gambar petunjuk di dalam ZIP menunjukkan cara memilih piksel untuk membaca flag sebenarnya dari bit terendah kanal merah.

Tahapan analisis dan penyelesaian

1. `file` mengidentifikasi `chall_2.png` sebagai PNG RGB 1024 × 1024. `exiftool` menampilkan `Comment: NCS{th3_w4ll_k33ps_s3cr3ts}` dan peringatan adanya data setelah `IEND`. Data tersebut berawalan `PK\x03\x04`, yaitu header ZIP. Arsip memuat `hint.png` dan meminta password; string pada `Comment` berhasil membukanya. Jadi, string metadata itu adalah kunci untuk tahap berikutnya.
2. Gambar `hint.png` berisi instruksi: *"Seek the red drops where the product of coordinates leaves a remainder of seven under thirteen."* Artinya, untuk setiap piksel dengan koordinat `(x, y)`, pilih piksel ketika `(x × y) % 13 == 7`. Kanal yang dimaksud adalah merah (*red*).
3. Mulai dari kiri atas, telusuri gambar per baris. Ambil bit terendah (`R & 1`) dari kanal merah pada setiap piksel terpilih, lalu gabungkan setiap delapan bit menjadi satu byte. Awal hasilnya adalah `NCS{t0ur1ng_th3_gr4ff1t1_m4z3_w1th_m4th}`. Data setelah `}` tidak termasuk flag.

Skrip reproduksi

Jalankan dengan `python3` dari folder yang berisi `chall_2.png`. Dependensi: Pillow (`pip install Pillow`). Skrip membaca PNG asli, mengambil ZIP, membuka `hint.png`, dan mengekstrak flag.

```
import io
import re
import struct
import zipfile
from pathlib import Path

from PIL import Image


png = Path("chall_2.png").read_bytes()
assert png.startswith(b"\x89PNG\r\n\x1a\n")

# Baca chunk PNG agar metadata dan awal data setelah IEND ditemukan.
offset = 8
password = None
trailer_start = None
while offset < len(png):
    length = struct.unpack_from(">I", png, offset)[0]
    kind = png[offset + 4 : offset + 8]
    data = png[offset + 8 : offset + 8 + length]
    offset += 12 + length
    if kind == b"tEXt" and data.startswith(b"Comment\x00"):
        password = data.split(b"\x00", 1)[1]
    if kind == b"IEND":
        trailer_start = offset
        break

assert password is not None and trailer_start is not None
print("Password ZIP:", password.decode())

with zipfile.ZipFile(io.BytesIO(png[trailer_start:])) as zf:
    hint_png = zf.read("hint.png", pwd=password)
```

```
assert hint_png.startswith(b"\x89PNG\r\n\x1a\n")
print("Petunjuk:", "(x * y) % 13 == 7, kanal merah")

with Image.open(io.BytesIO(png)) as image:
    image = image.convert("RGB")
    pixels = image.load()
    bits = [str(pixels[x, y][0] & 1)
            for y in range(image.height)
            for x in range(image.width)
            if (x * y) % 13 == 7]

bitstream = "".join(bits)
payload = bytes(int(bitstream[i:i + 8], 2)
                for i in range(0, len(bitstream) - 7, 8))
match = re.search(rb"NCS\{[^}]+\}", payload)
assert match is not None
print("Flag:", match.group().decode())
```

Keluaran verifikasi:

```
Password ZIP: NCS{th3_w4ll_k33ps_s3cr3ts}
Petunjuk: (x * y) % 13 == 7, kanal merah
Flag: NCS{t0ur1ng_th3_gr4ff1t1_m4z3_w1th_m4th}
```

Flag

```
NCS{t0ur1ng_th3_gr4ff1t1_m4z3_w1th_m4th}
```

chall_3 — Steganografi Labirin

Kompetisi: NCS 2026, Babak Final (29 September 2026)
Kategori: CRYPT
Berkas challenge: `chall_3.png`
Flag: `NCS{4lg0r1thm1c_st3g0_m4z3_m4st3r}`

Ringkasan

Gambar PNG 201 × 201 tampak seperti noise, tetapi bit terendah kanal birunya membentuk labirin. Jalur dari kiri atas ke kanan bawah menentukan urutan piksel yang harus dibaca. Bit terendah kanal merah pada jalur itu menghasilkan flag.

Tahapan analisis dan penyelesaian

1. `file` dan `exiftool` mengidentifikasi `chall_3.png` sebagai PNG RGB 201 × 201 tanpa metadata petunjuk atau data setelah `IEND`. Gambar terlihat acak. Dengan Python dan Pillow, bit terendah setiap kanal diperiksa secara terpisah. Pembacaan kanal merah secara berurutan per baris diawali `NCS{y0u_h1t_4_w4ll_try_4g...` lalu berubah menjadi noise. Kanal hijau mengulang `NCS{w4nd3r1ng_1n_th3_d4rk}`. Kedua hasil itu mengarahkan perhatian ke jalan keluar labirin.
2. Ketika bit terendah kanal biru divisualisasikan, nilai `1` membentuk koridor dan nilai `0` membentuk dinding. Titik awal koridor adalah `(1, 1)` dan titik tujuannya `(199, 199)`. Pencarian jalur dengan BFS menemukan rute sepanjang 3.813 piksel, termasuk kedua ujungnya.
3. Sepanjang rute tersebut, ambil `R & 1` dari setiap piksel dalam urutan jalan dari awal ke tujuan. Gabungkan setiap delapan bit dengan bit pertama sebagai bit paling signifikan. Byte awal hasilnya adalah `NCS{4lg0r1thm1c_st3g0_m4z3_m4st3r}`; data sesudah `}` tidak termasuk flag.

Skrip reproduksi

Jalankan dengan `python3` dari folder yang berisi `chall_3.png`. Dependensi: Pillow (`pip install Pillow`).

```
from collections import deque
from pathlib import Path
import re

from PIL import Image


with Image.open(Path("chall_3.png")) as image:
    image = image.convert("RGB")
    width, height = image.size
    assert (width, height) == (201, 201)
    pixels = image.load()

    start = (1, 1)
    finish = (199, 199)
    assert pixels[start][2] & 1
    assert pixels[finish][2] & 1

    queue = deque([start])
    previous = {start: None}
    while queue and finish not in previous:
        x, y = queue.popleft()
        for dx, dy in ((0, 1), (1, 0), (0, -1), (-1, 0)):
            nx, ny = x + dx, y + dy
            neighbor = (nx, ny)
            if (0 <= nx < width and 0 <= ny < height
                    and neighbor not in previous
                    and (pixels[nx, ny][2] & 1)):
                previous[neighbor] = (x, y)
                queue.append(neighbor)
```

```
    assert finish in previous
    path = []
    current = finish
    while current is not None:
        path.append(current)
        current = previous[current]
    path.reverse()
    print("Panjang jalur:", len(path))

    bits = "".join(str(pixels[x, y][0] & 1) for x, y in path)

payload = bytes(int(bits[i:i + 8], 2)
                for i in range(0, len(bits) - 7, 8))
match = re.search(rb"NCS\{[^}]+\}", payload)
assert match is not None
print("Flag:", match.group().decode())
```

Keluaran verifikasi:

```
Panjang jalur: 3813
Flag: NCS{4lg0r1thm1c_st3g0_m4z3_m4st3r}
```

Flag

```
NCS{4lg0r1thm1c_st3g0_m4z3_m4st3r}
```